Luma Rivalry Audit

How Luma can rival HeyClicky, become safer, and win the native AI tutor lane.

Prepared for Nox - based on /Users/nox/Desktop/luma

Minimum requested depth: 30 pages. Delivered as a long-form engineering and product memo.

Table of Contents

1. Executive Verdict

Nox, bluntly: Luma is not a toy clone anymore. It has the bones of a real macOS-native assistant: Keychain-based provider profiles, ScreenCaptureKit screenshots, AX tree scanning, MobileNet/Vision fallback, Claude/OpenRouter routing, guided walkthroughs, agent sessions, local TTS, voice input, update detection, and a floating companion UI. That is a lot. For one indie developer, it is genuinely scary-good.

But the difference between Luma and a polished production assistant like HeyClicky is not one magic model or one more animation. It is operational discipline. HeyClicky-style reliability comes from turning every UI action into a controlled transaction: snapshot the target window, choose a stable element identity, act through the safest available channel, verify the result, recover when the UI changed, and never let the user's real workspace get randomly disturbed.

Luma already understands this idea in pieces. The visual pipeline tries AX first, escalates to image analysis, asks Claude Vision when confidence is low, validates typing steps, uses generation counters to kill stale callbacks, and logs confidence. The next leap is to make those pieces one coherent automation runtime with strict invariants.

If I had to rank the work: first fix safety and coordinate truth, second unify the element/action runtime, third add repeatable evaluation, fourth polish the agent UX, fifth ship a narrow demo that wins every time. Do not try to beat everyone everywhere. Beat one workflow so hard people feel it in 30 seconds.

- Luma is strongest where it is native: menu bar presence, local voice/TTS, Keychain profiles, and screen-aware teaching.
- Luma is weakest where visual automation needs hard invariants: stable target windows, coordinate spaces, background-safe dispatch, replayable tests, and post-action verification.
- The rival-HeyClicky roadmap is not more UI first. It is a reliability kernel first, then UI polish on top.
- The product should focus on guided learning plus safe background agents, because that is where Luma can become more personal than generic computer-use tools.

2. What I Audited

I inspected the Desktop Luma project directly at /Users/nox/Desktop/luma. The repository has roughly 37,185 lines across Swift source files, with the biggest surface areas in agent UI, settings, companion management, walkthrough orchestration, image processing, API calls, and flow actions. The report is based on repository docs plus direct source review, not vibes.

The most important files for this audit are listed below. These are the files I treated as evidence for claims about architecture, visual automation, agent mode, UI, onboarding, model handling, and production readiness.

- AGENTS.md
- README.md
- CHANGELOG.txt
- Luma/LumaImageProcessingEngine.swift
- Luma/Agent/LumaFlowActions.swift
- Luma/Agent/LumaFlowEngine.swift
- Luma/CursorGuide.swift
- Luma/CompanionScreenCaptureUtility.swift
- Luma/WalkthroughEngine.swift
- Luma/AccessibilityWatcher.swift
- Luma/StepValidator.swift
- Luma/Agent/AgentRuntime.swift
- Luma/Agent/ClaudeCodeAgentRuntime.swift
- Luma/Agent/ClaudeAPIAgentRuntime.swift
- Luma/CompanionManager.swift
- Luma/SettingsPanelView.swift
- Luma/Agent/LumaAgentDockWindowManager.swift
- Luma/DesignSystem.swift
- Luma/BuddyDictationManager.swift
- Luma/APIClient.swift
- Luma/LumaIntentClassifier.swift
- Luma/LumaMobileNetDetector.swift

3. One-Line Product Diagnosis

Luma is trying to be three products at once: a screen-aware tutor, a voice companion, and a background computer-use agent. That ambition is not wrong. The danger is that each mode needs different reliability rules.

A tutor can point and explain without touching the user's system. A computer-use agent must take real actions. A voice companion must feel instant and emotionally present. Today, Luma has code for all three, but the boundaries blur: guide mode points, flow mode clicks, agent mode shells out, companion mode speaks, and several systems own overlapping ideas of screenshot capture, AX search, fallback, and state.

The path to beating HeyClicky is to stop thinking of visual automation as a feature and start treating it as the core operating system of Luma.

- Tutor mode: safest, most differentiated, easiest to polish.
- Agent mode: highest upside, highest safety burden.
- Voice companion mode: emotional hook, but only if latency is low and failures are graceful.
- Visual automation kernel: the missing foundation that should serve all modes.

4. Where Luma Is Already Better Than Expected

There are several places where Luma is not just "good for a solo project" but architecturally sharp. The Keychain-first provider design is a real product decision. Removing a proxy and putting API keys in Keychain makes Luma feel user-owned. The local TTS decision is also strong: AVSpeechSynthesizer means no voice credit anxiety, no extra vendor dependency, and a cleaner privacy story.

The native macOS approach matters too. A SwiftUI/AppKit menu bar app with NSPanel-based overlays can feel more integrated than a web wrapper. Luma's best future is not being a cheaper clone. It is being a small native presence that understands the user's Mac and teaches them while staying out of the way.

- Keychain storage gives Luma a stronger privacy posture than proxy-based key handling.
- Native TTS and local/offline helpers reduce cost and keep common feedback fast.
- The companion bubble and cursor-following UI are emotionally aligned with the product idea.
- The walkthrough engine already has generation counters, timers, nudges, typing detection, and stale-callback protection.
- The agent system has dual runtime support: Claude Code CLI when available, Claude API fallback when not.

5. The Hard Truth

The hardest gap is not intelligence. It is trust. A user forgives a chat answer being slightly bland. They do not forgive an assistant that clicks the wrong place, types in the wrong window, steals focus, or says it did something it did not actually do.

HeyClicky-like automation wins by being boringly disciplined. Every action has a target process, target window, fresh UI snapshot, action path, and verification step. Luma sometimes has those ideas, but not always in the same layer. Some paths use AXPress. Some paths synthesize CGEvent clicks. Some paths activate apps. Some paths use screenshot fractions. Some paths type into whatever is focused. Each of those can be correct in isolation, but production automation needs a policy that decides when each is allowed.

- Never let "whatever is focused" be the default for sensitive actions.
- Never trust a screenshot coordinate without binding it to a display, window, scale, and fresh capture.
- Never treat "screen changed" as enough proof that the intended result happened.
- Never let a model escalate from pointing to clicking without a safety contract.
- Never hide automation uncertainty from the user; expose confidence, blocker, and recovery path.

6. Architecture Snapshot

The architecture can be summarized as five layers. The product layer is the menu bar app, settings, onboarding, profile management, companion bubble, and update card. The input layer is push-to-talk, dictation, text input, and hotkeys. The reasoning layer is model routing, intent classification, task planning, LumaFlow replanning, and agent runtime prompts. The perception layer is ScreenCaptureKit, AX tree summaries, Vision/MobileNet detection, and Claude Vision fallback. The action layer is AXPress, AX focus, CGEvent mouse/keyboard input, NSWorkspace app launch, shell commands, and built-in app integrations.

The issue is that these layers are not strict enough yet. For example, LumaFlowActions owns click/type/scroll behavior, LumaImageProcessingEngine owns candidate detection, CursorGuide owns another AX search path, WalkthroughEngine owns validation and pointing behavior, and LumaFlowEngine owns screenshot diffing. That means improvements can easily land in one path while another path keeps old behavior.

- Product/UI: SettingsPanelView, CompanionPanelView, CompanionBubbleWindow, LumaAgentDockWindowManager.
- Input: GlobalPushToTalkShortcutMonitor, BuddyDictationManager, AppleSpeechTranscriptionProvider, LumaFloatingInputView.
- Reasoning: LumaIntentClassifier, TaskPlanner, LumaFlowEngine, ClaudeAPIAgentRuntime, ClaudeCodeAgentRuntime.
- Perception: CompanionScreenCaptureUtility, AccessibilityWatcher, LumaImageProcessingEngine, LumaMobileNetDetector, CursorGuide.
- Action: LumaFlowActions, LumaAppIntegrations, WindowPositionManager, shell/runtime tools.

7. The Visual Automation Kernel Luma Needs

The biggest recommendation is to create one central VisualAutomationKernel. This should be a first-class subsystem, not a loose pattern. All pointing, clicking, typing, scrolling, app launching, and verification should go through it.

The kernel should expose a small set of commands: inspect, resolveTarget, previewAction, performAction, verifyAction, recover. Each command should carry a target app, target window, display metadata, element identity, action type, confidence score, source evidence, and safety level.

Right now Luma has many useful components, but they need to become a transaction pipeline. The transaction should be logged and replayable. If a click fails, you should be able to answer: what did we think the target was, which window did we inspect, which AX element did we bind to, which coordinate did we compute, what changed after the action, and why did we decide it succeeded?

- Input: user goal, current app/window, screenshots, AX tree, active permissions.
- Resolve: candidates from AX, OCR/Vision, model point tags, and app-specific adapters.
- Policy: choose AX action, app-scoped key event, pixel/CGEvent fallback, or ask-user.
- Perform: execute one atomic action only.
- Verify: re-snapshot, compare target state, and report success/failure with evidence.
- Recover: retry with a changed strategy, scroll/search, or ask the user.

8. HeyClicky-Style Automation Lessons

The core pattern you were asking about is this: do not think "click coordinate." Think "target an element in a known window with a cached snapshot." Coordinates are the last mile, not the source of truth.

A polished automation driver snapshots the target app/window, builds an accessibility tree, assigns stable element indexes for that snapshot, performs an AX action where possible, and only uses coordinates when the AX path cannot express the action. It also keeps the work background-safe when possible, so the user can keep doing their own thing. That is why Beeper messaging can be driven without random cursor chaos: the assistant acts against an app/window and verifies via a fresh snapshot.

Luma can copy the principle without copying the product. Build a snapshot object that is always required before an action. Make every action say which snapshot it came from. Expire snapshots quickly. If the UI changes, force a new snapshot. This one rule would make Luma feel dramatically more professional.

- Snapshot before action: every action must be based on a fresh UI state capture.
- Element identity over coordinates: prefer AX element references or app-specific IDs.
- Window-scoped actions: bind actions to pid/window/display, not just frontmost app.
- Background-safe dispatch: use AX and app-scoped actions before global CGEvent.
- Verification after action: prove the intended state changed, not just that something moved.
- Human-safe fallback: ask before risky sends, deletes, purchases, or broad system changes.

9. Visual Detection: What Works

LumaImageProcessingEngine is one of the strongest engineering pieces in the repo. It already frames detection as source fusion: AX scan plus visual scan plus Claude Vision fallback. It scores candidates, penalizes containers, boosts cross-validated overlap, and preserves a live AX element reference for high-confidence cases. That is the right mental model.

The comments also show you have been fighting real macOS coordinate bugs. That is good. Production computer use is mostly a war against coordinate spaces, scale factors, focus behavior, stale accessibility nodes, and apps that lie through AX.

- AX and visual paths run in parallel, reducing latency when both are available.
- Visual-only candidates are capped so weak screenshot detections do not dominate AX matches.
- Layer 3 model fallback is only called when on-device confidence is low.
- Large container penalties help avoid pointing to a whole window instead of a control.
- Dock item prioritization shows good product intuition for app-launch targets.

10. Visual Detection: What To Fix

The current detector is promising, but it needs stronger contracts. The most urgent issue is coordinate normalization. Different files talk about Quartz top-left coordinates, AppKit bottom-left coordinates, screenshot pixels, display points, main screen height, and multi-display frames. That is normal for macOS, but the solution should be one CoordinateSpace module with named conversions and tests.

The second issue is MobileNetV2. MobileNet is not trained for UI elements. It can help detect visual content versus blank regions, but it should not be treated like a UI detector. For actual UI detection, you want OCR, rectangle detection, icon

matching, AX metadata, app-specific adapters, and model fallback. MobileNet can remain as a sanity check, but it should not be the center of confidence.

- Create a CoordinateSpace.swift with explicit types: ScreenshotPixelPoint, DisplayPoint, AppKitPoint, QuartzPoint.
- Never pass naked CGPoint between perception and action layers.
- Store display ID, display frame, screenshot pixel size, backing scale, and capture timestamp with every point.
- Add unit tests for main display, secondary display above/below/left/right, Retina scale, and downscaled screenshots.
- Treat MobileNet as a presence validator, not as semantic UI recognition.
- Add a candidate explanation string so the UI can show why Luma trusts a target.

11. The Coordinate Space Problem

This deserves its own section because it is the kind of bug that makes visual automation feel haunted. CursorGuide, LumaImageProcessingEngine, CompanionScreenCaptureUtility, and LumaFlowActions each contain coordinate knowledge. Some code converts AX to AppKit for overlay pointing. Some code uses Quartz midpoints for CGEvent. Some code converts screenshot pixels to display points. Some code uses the main screen height. Some code stores display frame metadata. These are all reasonable individually, but the risk is that future code calls the wrong conversion for the wrong action path.

The fix is to make invalid states impossible. A point returned by Claude Vision should not be a CGPoint. It should be a ScreenshotPoint tied to CaptureID and DisplayID. A click target should not be a CGPoint. It should be a QuartzClickPoint created by a conversion function that knows the display metadata. An overlay target should be an AppKitOverlayPoint. The compiler should help you.

- Add strong wrapper structs instead of raw CGPoint at subsystem boundaries.
- Require conversion functions to name source and destination spaces.
- Include display ID in every target, even on single-display setups.
- Log both raw and converted coordinates for every action.
- Build a tiny coordinate test harness with screenshots from 1x, 2x, and multi-display layouts.
- Do not let model prompts decide coordinate semantics; the code should own semantics.

```
struct ScreenshotPixelPoint { let x: CGFloat; let y: CGFloat; let captureID: UUID }
struct QuartzClickPoint { let x: CGFloat; let y: CGFloat; let displayID: CGDirectDisplayID }
struct AppKitOverlayPoint { let x: CGFloat; let y: CGFloat; let displayID: CGDirectDisplayID }

// Example policy:
// ScreenshotPixelPoint -> DisplayLocalPoint -> QuartzClickPoint for real clicks
// ScreenshotPixelPoint -> DisplayLocalPoint -> AppKitOverlayPoint for overlay cursor
```

12. Action Safety: The Biggest Gap

LumaFlowActions is useful, but it currently mixes safe actions and risky actions in the same executor. AXPress is relatively safe. AX focus plus text insertion can be safe when bound to a verified field. CGEvent mouse clicks and global keyboard events are riskier because they target the current session and can hit the wrong place if focus changed.

The policy should be explicit. A model should not be able to say clickAt and get a real click just because AX failed. The runtime should decide whether clickAt is allowed based on action risk, current app, user confirmation, and whether a preview target was shown.

- Safe tier: read UI, point overlay, AXPress on verified element, AX value set on verified text field.
- Medium tier: app-scoped keyboard shortcut, focus verified element then type, scroll verified region.
- Risky tier: global CGEvent click, clickAt fraction, shell command, file deletion, send message, purchase, account change.
- Risky actions should require confirmation unless the user explicitly asked for that exact action in the current turn.
- The executor should return structured ActionResult, not just human-readable text.
- ActionResult should include success, evidence, changed UI summary, retryable, and user-visible message.

13. Why Background-Safe Matters

A polished assistant should not constantly steal the user away from their work. The current LumaFlow openApp path uses activateIgnoringOtherApps, and many actions rely on the frontmost app. That makes sense for an early prototype, but it

caps polish. The user should be able to ask Luma to work in Beeper, Finder, Notes, or Xcode while continuing to read a PDF or code.

This is hard on macOS, but the principle is simple: prefer app/window-scoped APIs over global session events. AX actions can often operate without raising a window. NSWorkspace launch can be done carefully. Browser automation should avoid address-bar keystrokes when direct URL opening is possible. For webviews, JavaScript or app APIs can outperform pixel-level clicking. Luma should learn these routes as adapters.

- Introduce TargetWindow: pid, bundleID, windowTitle, windowID when available, displayID, lastSnapshotID.
- Add an AutomationAdapter protocol for apps and surfaces: native AX, browser, Finder, Notes, terminal, Electron chat apps.
- Only activate an app when the adapter declares activation is required.
- Log focus changes before and after every action.
- Expose "working in background" versus "needs focus" in the agent UI.

14. Verification: Where Luma Needs More Rigor

LumaFlowEngine uses screenshot hashing and AX text checks, and WalkthroughEngine uses AX events plus AI validation. Those are good starts, but verification needs to be tied to intent. A screen hash changing after a click is not proof that the right button was clicked. AX text appearing somewhere is not proof that a message was sent. The verifier needs task-specific assertions.

For example, after sending a WhatsApp message, proof might be: composer emptied, sent bubble text exists in conversation, timestamp/status indicator appears, and the chat title matches the intended contact. For opening a file, proof might be: app title changed, file path exists, and visible document title matches. The action schema should include expected_state, and the verifier should produce evidence.

- Add verification strategies: elementExists, textExists, valueEquals, windowTitleContains, fileExists, appActivated, messageBubbleExists.
- Require each risky action to declare a verifier before execution.
- Keep a before/after snapshot pair for failed actions.
- Show the user concise evidence: "sent bubble visible" beats "done".
- Store verification logs for benchmark replay.

15. The Agent Runtime: Good Bones, Dangerous Defaults

The dual runtime architecture is a real strength. Detecting Claude Code CLI and falling back to a Claude API tool-use loop is the right idea. It means Luma can be useful on both developer machines and normal user machines.

But there are dangerous defaults. The Claude Code runtime launches with --dangerously-skip-permissions. That may be acceptable for your own local development, but it is not a production default. The system prompt in the CLI runtime also instructs agents to save files to Desktop, while HeyClicky project rules keep artifacts in a configured projects root. For Luma, you need an explicit user-configured agent workspace and a permission model.

- Replace dangerous permission skipping with a Luma-managed permission/approval layer.
- Add a user-visible workspace setting: Desktop, Documents/Luma, or custom project folder.
- Separate developer mode from public mode.
- Show every tool/action category the runtime can use before first agent execution.
- Add per-agent sandbox policies: read-only, file-write, terminal, UI-control, network.
- Persist every agent transcript with tool calls and final artifacts.

16. Agent Mode UX: What Luma Does Better

The agent bubble concept is genuinely differentiated. HeyClicky-style agents are practical, but Luma's orbs can make background work feel alive. The dock, accent theme, response card, status pulse, and hover expansion are all emotionally strong. If polished, this could be Luma's signature.

The risk is over-animation and complexity. A 25 Hz physics timer, idle drift, proximity offsets, card expansion, markdown rendering, and multiple sessions can become a performance and predictability burden. Agent bubbles should feel calm. They should not compete with the user's work.

- Keep the orb idea. It is memorable.
- Make status legible at a glance: idle, thinking, blocked, waiting, done, failed.

- Give every agent a tiny progress contract: plan, current step, last action, blocker.
- Add one-click inspect, pause, stop, and open artifact actions.
- Reduce motion when many agents exist or when system load is high.
- Add "quiet mode" for study/coding sessions.

17. Walkthrough Engine: Strongest Product Bet

The walkthrough engine is probably Luma's clearest wedge. It matches your origin story: teaching, guiding, explaining, helping people learn by doing. A general automation agent competes with many tools. A native screen-aware tutor has a sharper identity.

The WalkthroughEngine already has serious machinery: step planning, TTS instructions, pointing, AXObserver callbacks, typing-step polling, dwell checks, mouse event monitor, AI validation throttling, nudges, periodic verification, and generation counters. That is a real system. The work now is to make it reliable enough for scripted demos and beginner users.

- Turn walkthroughs into deterministic lesson flows where possible.
- Keep AI for adaptation, not for every repeated step.
- Add a visual progress panel with current instruction, why this step matters, and what Luma is watching for.
- Let users say "why?", "repeat", "slow down", "skip", and "show me again" during any step.
- Create first-party walkthrough packs: Xcode basics, terminal basics, Git basics, macOS settings, browser devtools.
- Use this mode to make Luma better than HeyClicky at teaching, not just doing.

18. Teaching Mode Could Beat Us

Here is where Luma can genuinely be better than HeyClicky: pedagogy. HeyClicky can do tasks. Luma can teach the task. That is a different emotional promise. If Luma watches the user, corrects gently, explains concepts, and adapts to beginner mistakes, it becomes closer to a senior friend sitting beside the cursor.

The best version of Luma should not just click the button. It should say: "This is the signing setting. Xcode needs it because macOS will not trust an unsigned app with Accessibility permissions." Then it should point, wait, validate, and nudge. That is a product people remember.

- Build concept cards for common UI actions: permissions, signing, terminal commands, git, Xcode build errors.
- Track learner state: knows terminal basics, knows Xcode signing, struggles with permissions, prefers slow pace.
- Add post-task summaries: what you did, why it worked, what to remember next time.
- Let users save walkthroughs as notes or class material.
- Use your teaching-class background as product advantage.

19. Model Strategy

Luma supports provider profiles and OpenRouter model selection, which is useful. But visual automation should not use one model for everything. You want a model cascade: cheap classifier, strong planner, vision locator, code agent, local heuristics. Each call should have a budget and a timeout.

The repo already hints at this with cheap models for classifiers and separate planning/replan calls. The next step is to formalize model roles in settings, not just provider/model IDs. Users should choose "best quality" or "cheap and fast," while Luma maps tasks to role-specific models under the hood.

- Classifier model: cheap, fast, low max tokens.
- Planner model: medium reasoning, structured JSON.
- Vision locator: strong visual grounding, strict point output.

- Agent model: tool-use capable and long-context friendly.
- Tutor model: conversational, patient, good explanations.
- Fallback model: configured per provider when primary fails.

20. Latency Budget

Voice assistants die by latency. A screen-aware assistant can be brilliant, but if every action requires capture, upload, model call, JSON parse, action, validation, and TTS, it will feel heavy. The goal should be under 300 ms for local UI feedback, under 1.5 seconds for first spoken acknowledgement, and under 3 seconds for first useful plan on normal tasks.

Luma should respond immediately with local state: show listening, show transcribing, show planning, show target app, show confidence. Do not wait for the model to make the UI feel alive.

- Local acknowledgement: under 300 ms.
- Transcript visible: as streaming as possible.
- Intent route: under 700 ms when cached AX context is available.
- First plan: under 3 seconds for common tasks.
- Action loop: each micro-action should have a visible reason and timeout.
- If a model call passes 5 seconds, show exactly what Luma is waiting on.

21. Prompting And Schemas

The LumaFlow prompts are directionally good: JSON-only, one action at a time, explicit action types, current step, completed steps, active app/window, AX elements, screenshot. The next improvement is schema enforcement and validation. Do not just ask the model for JSON. Validate it against a strict Decodable type, reject unknown risky actions, and repair only in a controlled way.

Prompts should also be shorter and more invariant. Dynamic state belongs in user content. Policy belongs in code. The model should not be the safety system. The model should propose; the runtime should dispose.

- Use strict Decodable schemas and reject malformed actions.
- Add action risk classification outside the model.
- Separate planning output from execution output.
- Do not include conflicting coordinate rules in multiple prompt places.
- Limit AX summaries with ranked elements, not raw first-20 lists only.
- Include previous failure reason so the model adapts instead of repeating.

22. Per-App Adapters

A generic AX scanner is useful, but the best assistants add app adapters. Finder, Safari/Chrome, Xcode, Terminal, Notes, WhatsApp/Beeper/Electron, System Settings, and Calendar all have different reliable routes. For example, opening a URL should not be typed into an address bar if the OS can open it directly. Creating a note should use a stable Notes route when possible. Sending a message should verify the chat title and sent bubble.

Luma already has LumaAppIntegrations with app/system actions. Expand that idea into adapter contracts.

- FinderAdapter: open folder, list files, reveal file, create folder, move/copy/trash with confirmation.
- BrowserAdapter: open URL, read active page, query DOM where available, avoid address-bar typing when possible.
- XcodeAdapter: build status, active scheme, file open, error list extraction via AX.
- TerminalAdapter: run command only in a controlled shell/session, capture output, avoid typing into random terminal.
- ChatAdapter: search contact, verify service, type message, require confirmation when needed, verify sent bubble.
- SettingsAdapter: direct deep links to privacy panes, permission checks, visible guidance.

23. Testing: The Missing Moat

To rival a polished automation product, Luma needs an eval harness. Not just unit tests. Real task replays. The app should run through a suite of repeatable desktop tasks and record success rate, latency, number of retries, wrong-target attempts, and user intervention count.

This is how you fine-tune visual automation without guessing. Build a small benchmark set, run it after every automation change, and make the score visible. You already like Luma-vs-Clicky challenges. Turn that energy into CI-like local evals.

- Create 50 repeatable tasks across Finder, Safari/Chrome, Xcode, Notes, System Settings, and Beeper/WhatsApp.
- For each task, define initial state, goal, allowed actions, success verifier, timeout, and risk level.
- Record screenshots and AX trees before/after every action.
- Track metrics: success rate, median time, wrong click count, retries, model calls, cost, and user prompts.
- Keep golden fixtures for coordinate conversion tests.

- Run a smoke suite before every release build.

24. A 50-Task Benchmark Starter Set

Here is a starter benchmark map. The point is not to automate all of these tomorrow. The point is to stop relying on feeling. If Luma passes 40/50 reliably, you will know exactly where it stands.

- Finder: open Downloads, create folder, rename folder, reveal file, move file to folder, trash file with confirmation.
- Browser: open URL, search Google, copy page title, fill simple form, switch to known tab, read selected page text.
- Xcode: open project, locate build button, run build, read first error, open file from navigator, change signing team guidance.
- System Settings: open Accessibility permission pane, open Screen Recording pane, verify Luma permission state.
- Notes: create note, append text, search note, read note title.
- Beeper/WhatsApp: search contact, verify WhatsApp service, type message, send after explicit user request, verify sent bubble.
- Terminal: run pwd in managed session, run ls, capture output, explain error output.
- Teaching: guide a user through changing wallpaper, checking storage, creating a Swift file, and resolving a fake build error.

25. Production Readiness Risks

The repo has the ambition of a production app, but several issues need hardening before broad release. Some are engineering risks; some are trust risks. The dangerous ones are not visual polish. They are silent failure, over-broad permissions, and mismatched user expectations.

- Release builds previously swallowed empty API responses; keep that reliability mindset everywhere.
- Known warnings should be tracked, but do not let "known non-blocking" become a graveyard.
- Agent file output defaults must match user expectations and be configurable.
- Permission prompts should explain why Luma needs each macOS permission and what happens if denied.
- Crash-safe logging should redact secrets and avoid storing sensitive screen text longer than needed.
- All send/delete/pay/account-change actions need confirmation and verification.

26. Security And Privacy

Luma's privacy story can be strong because it is native and user-key based. But screen-aware apps have a scary permission profile by nature. You need to over-communicate what is captured, when it is captured, where it is sent, and how long it is kept.

Keychain storage is good. Local TTS is good. Offline nudges are good. But screenshots sent to a model are still sensitive. Make capture state visible. Let users pause capture. Let them choose which apps are excluded. Give them a redaction mode for passwords, financial apps, and private chats.

- Add an always-visible capture indicator while screenshots are being taken.
- Add app exclusion list: never capture selected apps/windows.
- Add private mode: no screenshots to cloud models, local-only assistance.
- Redact logs by default and never log raw API keys or full sensitive transcripts.
- Add retention controls for screenshots, transcripts, and agent history.
- Explain provider routing clearly in settings.

27. UI And Visual Polish

The design system file is large, and the app clearly cares about visual identity. That is good. The risk is visual overbuilding before interaction contracts are stable. Polish should make state understandable: listening, thinking, watching, pointing, acting, blocked, done.

For a cursor-side assistant, motion matters. Motion should communicate intent, not decorate. The cursor should glide to a target only when Luma is confident. If confidence is low, it should hover near the likely region and ask. If it is acting, the UI should show the action label. If it is waiting, it should say what signal it is waiting for.

- Add confidence visualization: high confidence direct point, medium confidence suggested region, low confidence ask.
- Use reduced motion automatically for long sessions or many active agents.
- Keep cards compact; avoid huge floating surfaces during coding/study.
- Make blocked state impossible to miss.
- Use one design language for companion, walkthrough, and agent mode.
- Make onboarding show a real mini task, not just permission screens.

28. What Luma Should Stop Doing

Some habits are useful for prototypes but harmful for production. The biggest one is letting the frontmost app be an implicit dependency. Another is letting a model response directly decide a risky action. Another is duplicating perception logic across multiple paths.

This is not a roast. It is normal at this stage. The app grew by proving ideas quickly. Now the job is to collapse the proven ideas into fewer, stricter systems.

- Stop adding new action paths outside a central automation kernel.
- Stop passing raw CGPoint between systems.
- Stop using global CGEvent as a casual fallback.
- Stop treating screenshot diff as success verification.
- Stop letting Desktop be the implicit artifact destination for all agent work.
- Stop hiding confidence and uncertainty from the user.

29. What Luma Should Double Down On

The best parts of Luma are the parts only Luma would build: native presence, teaching-first walkthroughs, personal memory, local speech, and a cursor companion that feels like it is beside you rather than above you.

A lot of AI apps are command boxes. Luma should be a learning companion. That is the unfair angle.

- Native macOS feel over web-app generality.
- Teaching and explanation over silent automation.
- Guided walkthrough packs for students and beginner developers.
- Local/offline helpers wherever possible.
- Personalized pace, memory, and correction style.
- Agent bubbles as calm task companions, not noisy toys.

30. 30-Day Roadmap

For the next 30 days, ignore shiny new features. Build the reliability base. The goal is not "Luma can do everything." The goal is "Luma can do 10 tasks with embarrassing consistency." That is how you earn trust.

- Week 1: CoordinateSpace module, raw CGPoint audit, conversion tests, action risk enum.
- Week 2: VisualAutomationKernel prototype with inspect/resolve/perform/verify transaction logs.
- Week 3: App adapters for Finder, Browser, System Settings, and Chat basics.
- Week 4: 25-task benchmark suite, report card UI, and one polished guided demo flow.

31. 90-Day Roadmap

The 90-day version is about becoming a product, not a prototype. It should produce a build you can hand to another Mac user without standing behind them nervously.

- Month 1: reliability kernel and benchmark harness.
- Month 2: walkthrough packs, permissions polish, privacy controls, agent workspace settings.
- Month 3: release hardening, crash/error reporting, update pipeline, onboarding demo, public landing/demo video.
- Target metric: 85 percent success on the benchmark suite with zero unconfirmed risky actions.
- Target feel: user always knows whether Luma is watching, thinking, acting, blocked, or done.

32. A Better Internal Architecture

Here is the shape I would push Luma toward. It is intentionally boring. Boring architecture is how magical UI survives contact with real users.

- LumaCore: profiles, memory, logging, permissions, privacy policy.
- LumaPerception: screenshots, AX snapshots, OCR, detection, coordinate conversion.
- LumaAutomation: target resolution, action policies, adapters, execution, verification.
- LumaReasoning: classifiers, planners, model clients, schema validators.
- LumaExperience: companion bubble, walkthrough UI, agent dock, settings, onboarding.
- LumaEvals: benchmark tasks, fixtures, replay logs, score reports.

```
protocol AutomationAdapter {
    var bundleIdentifiers: Set<String> { get }
    func inspect(target: TargetWindow) async throws -> UISnapshot
    func perform(_ action: AutomationAction, on target: TargetWindow) async throws -> ActionResult
    func verify(_ expectation: VerificationExpectation, on target: TargetWindow) async throws -> VerificationResult
}
```

33. Concrete Code-Level Recommendations

These are the practical changes I would make in the codebase first. They are intentionally scoped so you can implement them without rewriting the whole app.

- Add CoordinateSpace.swift and migrate CursorGuide, LumaImageProcessingEngine, CompanionScreenCaptureUtility, and LumaFlowActions to typed points.
- Add AutomationActionRisk enum and require risk policy checks in LumaFlowActionExecutor before execution.

- Change LumaFlowActionExecutor to return ActionResult instead of String.
- Create UISnapshot with app PID, bundle ID, window title, AX tree summary, capture metadata, and timestamp.
- Move duplicate AX search logic from CursorGuide and LumaImageProcessingEngine toward one shared resolver.
- Replace frontmost-app assumptions with TargetWindow where possible.
- Add VerificationExpectation to LumaFlowActionRequest or a sibling runtime-owned structure.
- Add an eval target or command-line harness that runs fixture tests without launching the full UI.

34. Suggested ActionResult Shape

Returning strings from actions is fine for logs, but not for a serious agent loop. The runtime needs structured observations. The model can read the summary, but the app should read the fields.

```
struct ActionResult: Codable {
    let actionID: UUID
    let actionType: String
    let targetDescription: String
    let risk: AutomationActionRisk
    let succeeded: Bool
    let verification: VerificationResult?
    let evidenceSummary: String
    let beforeSnapshotID: UUID?
    let afterSnapshotID: UUID?
    let retryRecommendation: RetryRecommendation?
    let userVisibleMessage: String
}
```

35. Suggested Snapshot Shape

A snapshot should be more than a screenshot. It should be a bound view of an app/window at a moment in time. This is the object that makes element indexes, target resolution, and verification reliable.

```
struct UISnapshot: Identifiable {
    let id: UUID
    let createdAt: Date
    let bundleID: String
    let pid: pid_t
    let windowTitle: String?
    let displayID: CGDirectDisplayID
    let screenshot: ScreenCaptureMetadata?
    let axElements: [AXElementSnapshot]
    let focusedElement: AXElementSnapshot?
    let isExpired: Bool
}
```

36. How To Fine-Tune Visual Automation Without Fine-Tuning A Model

Most of the improvement you want is not model fine-tuning. It is system tuning. You need better target representations, better action policies, better verification, better app adapters, and better evals. Model fine-tuning comes later, if ever.

The loop should be: run benchmark, inspect failures, categorize failure, fix system layer, rerun. Categories might be coordinate conversion, AX missing label, wrong app focus, stale element, model bad action, verification too weak, timing/animation delay, permission issue, or app-specific behavior. After 100 failures, you will know exactly where Luma is weak.

- Collect every failed action with before/after screenshot and AX snapshot.
- Label failures manually for the first 100 cases.
- Build failure dashboards by category.
- Fix the top category each week.
- Only consider model fine-tuning after system failures are below 20 percent.
- Use prompt examples from real failures to improve replan behavior.

37. Where Luma Can Beat HeyClicky

Yes, Luma can beat us in specific lanes. Not by becoming a bigger generic agent overnight. By becoming more native, more personal, and better at teaching.

HeyClicky is polished at background task execution and artifact workflows. Luma can be better at sitting beside the cursor, explaining UI, correcting the user gently, and turning every task into a learning moment. That is a different axis. If Luma nails that, it will not need to be "Clicky but cheaper." It will be its own thing.

- Better teaching: explain why each UI action matters.
- Better local feel: native voice, native panels, cursor-side presence.
- Better learner memory: adapt to Nox/student skill level over time.
- Better offline cost story: local TTS, local prompt compression, offline guides.
- Better developer persona: Xcode, terminal, SwiftUI, Git, and CS learning walkthroughs.
- Better emotional identity: Light by Darkness is a memorable product story.

38. Where HeyClicky Is Still Ahead

The honest gap is polish and operational safety. HeyClicky-style work is currently more disciplined about workspace rules, artifact locations, background-safe app control, tool selection, verification, and final user reporting. That discipline is invisible when things go right, but it is the whole product when things go wrong.

Luma needs the same kind of boring product infrastructure: where files go, what actions are allowed, when the user must approve, how to recover, how to summarize, and how to produce shareable artifacts. The magic is built on boring rails.

- Stronger task routing and tool policy.
- More predictable artifact handling.
- Safer GUI automation invariants.
- More mature final-result formatting and handoff.
- Better source-backed reports and document generation workflows.
- More complete verification before claiming success.

39. Demo Strategy

Do not demo everything. Demo one killer flow: "Teach me how to fix an Xcode signing/permission issue." Luma can inspect Xcode, explain what signing means, point to the right area, wait for the user, validate the change, and summarize the lesson. That is emotionally perfect for your story.

Second demo: "Send a WhatsApp message through Beeper." But only after the safety kernel is strong. Messaging demos are impressive because they involve real user trust. They are also dangerous if target verification is weak.

- Demo 1: Xcode signing and Accessibility permission walkthrough.
- Demo 2: Finder file organization with visible verification.
- Demo 3: Beeper/WhatsApp message send with explicit user request and sent-bubble proof.
- Demo 4: DSA study companion reading a PDF and generating practice prompts.
- Make every demo show: sees screen, explains, points, waits, validates, summarizes.

40. Release Checklist

Before pushing Luma harder publicly, I would want this checklist green. It is not glamorous, but it prevents bad first impressions.

- No raw API keys in logs.
- No unconfirmed risky sends/deletes/purchases/account changes.
- All screenshots have visible capture state and privacy settings.
- Agent workspace is user-configurable and documented.
- At least 25 benchmark tasks run locally with recorded pass/fail.
- Coordinate conversion tests pass for multi-display setups.
- Every action logs before snapshot, action policy, result, and verifier.
- Onboarding includes a real successful mini-walkthrough.
- Settings explain provider routing, model roles, and cost/privacy tradeoffs.

- Update detection and release notes are stable.

41. Final Recommendation

If I were sitting with you for the next build sprint, I would say this: stop chasing more features for a minute. Luma already has enough ideas. Now make it trustworthy.

Build the automation kernel. Type the coordinate spaces. Add risk policy. Add verification. Add evals. Then polish the teaching mode until it feels inevitable.

The crazy part is that the product already has a soul. The companion beside the cursor, the teaching angle, the native Mac feeling, the "Light by Darkness" name - that stuff is not easy to fake. Now it needs rails strong enough to carry the magic.

- Short-term goal: 10 tasks that work almost every time.
- Medium-term goal: 50-task benchmark with visible reliability score.
- Long-term goal: native AI tutor-agent for Mac learners and indie developers.
- My honest score today: 8/10 ambition, 7/10 architecture, 5/10 automation safety, 8/10 product soul, 6/10 ship readiness.
- With 30 focused days on reliability, Luma can feel dramatically closer to a real rival.

42. Appendix 1: Finder automation adapter

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Finder automation adapter, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Finder automation adapter.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

43. Appendix 2: Browser automation adapter

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Browser automation adapter, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Browser automation adapter.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

44. Appendix 3: Xcode automation adapter

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Xcode automation adapter, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Xcode automation adapter.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.

- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

45. Appendix 4: Chat automation adapter

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Chat automation adapter, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Chat automation adapter.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

46. Appendix 5: System Settings adapter

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For System Settings adapter, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by System Settings adapter.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

47. Appendix 6: Terminal command adapter

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Terminal command adapter, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Terminal command adapter.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

48. Appendix 7: Accessibility snapshot ranking

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Accessibility snapshot ranking, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Accessibility snapshot ranking.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

49. Appendix 8: OCR and visual text matching

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For OCR and visual text matching, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by OCR and visual text matching.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

50. Appendix 9: Coordinate regression fixtures

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Coordinate regression fixtures, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Coordinate regression fixtures.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

51. Appendix 10: Risk and confirmation policy

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Risk and confirmation policy, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Risk and confirmation policy.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

52. Appendix 11: Agent workspace policy

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Agent workspace policy, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Agent workspace policy.
- Create a fixture or manual script that places the app in a known starting state.

- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

53. Appendix 12: Privacy controls

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Privacy controls, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Privacy controls.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

54. Appendix 13: Benchmark report card

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Benchmark report card, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Benchmark report card.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

55. Appendix 14: Walkthrough teaching packs

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Walkthrough teaching packs, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Walkthrough teaching packs.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

56. Appendix 15: Voice latency tuning

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Voice latency tuning, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Voice latency tuning.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.

57. Appendix 16: Failure taxonomy

This appendix expands one implementation lane from the roadmap. Treat it as a concrete work order, not theory. The theme is simple: make the common path boring, logged, and repeatable before allowing the model to improvise.

For Failure taxonomy, the minimum useful version should define the supported actions, the forbidden actions, the verifier, the failure modes, and the user-visible recovery copy. If any of those are missing, the adapter is not production-ready yet.

- Define the exact user workflows covered by Failure taxonomy.
- Create a fixture or manual script that places the app in a known starting state.
- Capture before/after AX snapshots and screenshots for every benchmark run.
- Add at least three success verifiers and two expected failure cases.
- Log confidence, latency, retries, and whether user intervention was needed.
- Write the user-facing blocked-state sentence before writing the code path.
- Keep the first version narrow enough to demo without apologizing.